

What Makes Software Good?



Mike Bostock

Follow

21 min read · Mar 10, 2016



2.3K



34



As someone who creates open-source software, I spend a lot of time thinking about how to make software better.

This is unavoidable: there's an unending stream of pleas for help on Stack Overflow, in GitHub issues and Slack mentions, in emails and direct messages. Fortunately, you also see people succeed and make fantastic things beyond your imagination, and knowing you helped is a powerful motivation to keep at it.

So you wonder: what qualities of software lead people to succeed or fail? How can I improve my software and empower more people to be successful? Can I articulate any guiding principles, or do I just have intuition that I apply on a case-by-case basis? (Thinking about something and externalizing — articulating — that thought are two very different activities.) Perhaps something like [Dieter Ram's principles](#) for good design, tailored for software?

Good design is innovative.

Good design makes a product useful.

Good design is aesthetic.

Good design makes a product understandable.

Good design is unobtrusive.

Good design is honest.

Good design is long-lasting.

Good design is thorough down to the last detail.

Good design is environmentally-friendly.

Good design is as little design as possible.

Programming interfaces are user interfaces.

Or, to put it another way: Programmers are people, too.

Form must communicate function.

Functions that take many arguments are obviously
bad design.

```
export function mountChilds(childVas:HTMLCanvasElement){
```

Labels.

options: {

typre: "line",

3

1

data: {}





responsible: true,

```
data_sets: [{data, border_color: "#2563eb"}]
```

maintainAspectRatio: false,

```
return new Chart(canvas, {
```

`pubgins:: { legend:: { display: false } },`

Programming interfaces.

Or, put it another way: Programs are people, too.



Formulate function.

Functions that take many arguments are obviously

boarddesign.

